CHAPTER 6 — SOFTWARE METRICS

Multiple-Choice Questions (MCQ)

Section 1: Official Course-Slide MCQs (20 Items)

Q6.1. Which one is not a category of software metrics?

(a) Product metrics · (b) Process metrics · (c) Project metrics · (d) People metrics

(d) People metrics

In K.K. Aggarwal & Yogesh Singh's taxonomy (companion slides 9–10, 96; scan p. 254), software metrics are categorized into three primary classes:

1. **Product metrics: Measures of the software product at any development stage, ranging from requirements documents to source code and test suites** (e.g., size, complexity, reliability).
2. **Process metrics: Measures of the software development and maintenance process** (e.g., defect removal rate, development cycle duration, phase containment efficiency).
3. **Project metrics: Measures of project operational performance and resource utilization** (e.g., staffing allocation, cost tracking, schedule adherence, productivity).

"People metrics" is not one of the three established core categories in Aggarwal's classification.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 9–10, 96; scan p. 254.

Q6.2. Software science measures are developed by:

(a) M.Halstead · (b) B.Littlewood · (c) T.J.McCabe · (d) G.Rothermal

(a) M.Halstead

Software Science (also known as Halstead Metrics) was developed and published by Maurice H. Halstead in 1977. The theory operates on the premise that any computer program is a collection of tokens categorized strictly into operators and operands.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 255; companion slides 11, 96.

Q6.3. Vocabulary of a program is defined as:

(a) $\eta = \eta_1 + \eta_2$ · (b) $\eta = \eta_1 \times \eta_2$ · (c) $\eta = \eta_1 - \eta_2$ · (d) $\eta = \eta_1 / \eta_2$

(a) $\eta = \eta_1 + \eta_2$

The program vocabulary η represents the total number of distinct tokens appearing in a program. It is computed as the sum of:

- η_1**: Number of unique (distinct) operators.**
- η_2**: Number of unique (distinct) operands.**

Thus, $\eta = \eta_1 + \eta_2$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 256; companion slides 12, 96.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- ميز بدقة بين المفردات (Vocabulary η) وهو مجموع المعاملات والرموز الفريدة ($\eta_1 + \eta_2$)، وبين طول البرنامج (Length N) وهو مجموع التكرارات الكلية ($N_1 + N_2$).

Q6.4. In Halstead theory of software science, volume is measured in bits. The bits are:

(a) Number of bits required to store the program · (b) Actual size of a program if a uniform binary encoding scheme for vocabulary is used · (c) Number of bits required to execute the program · (d) None of the above

(b) Actual size of a program if a uniform binary encoding scheme for vocabulary is used

In Halstead's Software Science, Program Volume $V = N \log_2 \eta$ measures the information content or storage size of a program implementation.

Each of the N total tokens in the program requires a minimum binary representation of $\log_2 \eta$ bits (the minimum number of bits needed to distinguish among the η unique tokens in the program vocabulary under a uniform binary encoding). Therefore, the theoretical physical unit of Volume is **bits**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 257; companion slides 14, 96.

Q6.5. In Halstead theory, effort is measured in:

(a) Person-months · (b) Hours · (c) Elementary mental discriminations · (d) None of the above

(c) Elementary mental discriminations

Halstead defined mental effort E as the total number of elementary mental discriminations (decisions) required to develop or comprehend a program. It is calculated as:

$$E = \frac{V}{L} = D \times V$$

Where D is the program difficulty ($D = 1/L = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$) and V is the volume in bits.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 259; companion slides 19, 97.

Q6.6. Language level is defined as:

(a) $\lambda = L^3 V$ · (b) $\lambda = LV$ · (c) $\lambda = LV^*$ · (d) $\lambda = L^2 V$

(d) $\lambda = L^2 V$

Language level λ is an invariant attribute of a programming language, representing the expressive power and conciseness of the language. It is derived from the relationship:

$$\lambda = L \times V^*$$

Since the program level is defined as $L = V^*/V$, substituting $V^* = L \times V$ yields:

$$\lambda = L \times (L \times V) = L^2 V$$

Higher-level programming languages possess higher values of λ.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 260; companion slides 23, 97.

Q6.7. Program weakness is:

(a) $WM = \overline{LV} \times \gamma$ · (b) $WM = \overline{LV} / \gamma$ · (a) $WM = \overline{LV} + \gamma$ · (d) None of the above

(d) None of the above [literal answer to the printed stem]

> [!WARNING] **Source-item defect:** The printed stem asks for **Program Weakness (WP)**, but choices (a)–(c) give formulas for **Module Weakness (WM)**; the source also duplicates the label `(a)` on its third formula choice. Since no choice gives the program-level formula, `(d)` None of the above is the literal answer to the stem as printed. If the intended stem was "Module weakness," the formula in (a) is the correct one: $WM = \overline{LV} \times \gamma$. Program weakness is the mean of module weaknesses: $WP = \frac{1}{m} \sum_{i=1}^{m} WM_i$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 278–280, 305; companion slides 68, 97.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- هذا السؤال يحمل خطأً مطبعياً ومفاهيمياً في أصل شريحة الناشر والكتاب (شريحة 97): سأل عن ضعف البرنامج (Program Weakness)، لكن جميع الخيارات تعرض صيغ ضعف الوحدة (WM).
- ضعف الوحدة هو: $WM = \overline{LV} \times \gamma$ حيث $\overline{LV}$ هو متوسط المتغيرات الحية لكل تعليمة تنفيذية، وγ هو متوسط عمر المتغيرات.
- ضعف البرنامج الإجمالي هو متوسط ضعف وحداته: $WP = \frac{1}{m} \sum WM_i$.

Q6.8. "FAN IN" of a component A is defined as:

(a) Count of the number of components that can call, or pass control, to component A · (b) Number of components related to component A · (c) Number of components dependent on component A · (d) None of the above

(a) Count of the number of components that can call, or pass control, to component A

Fan-in of a component A is the count of the number of components that can call, or pass control, to component A. It measures how many other components depend on component A.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 282; companion slides 69, 97.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- الـ Fan-in هو عدد الوحدات التي تستدعي الوحدة (تدفق داخل إليها). ارتفاعه دليل على إعادة استخدام الوحدة (High Reuse)، ولكنه يعني أيضاً حساسية عالية لأن أي عطل فيها يؤثر على كل من يستدعيها.

Q6.9. "FAN OUT" of a component A is defined as:

(a) Number of components related to component A · (b) Number of components dependent on component A · (c) Number of components that are called by component A · (d) None of the above

(c) Number of components that are called by component A

Fan-out of a component A is the number of components that are called by component A. It reflects the control dispersion and coordination responsibility of the component.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 282; companion slides 69, 98.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- الـ Fan-out هو عدد الوحدات الفرعية التي تستدعيها هذه الوحدة (تدفق خارج منها). ارتفاعه الشديد يشير إلى أن الوحدة تتحمل تعقيد تحكم وتنسيق مفرط وتحتاج إلى إعادة هيكلة (Refactoring).

Q6.10. Which is not a size metric?

(a) LOC · (b) Function count · (c) Program length · (d) Cyclomatic complexity

(d) Cyclomatic complexity

Cyclomatic Complexity is a control-flow structural complexity metric formulated by Thomas McCabe, designed to quantify procedural logic and the number of linearly independent paths. It is not a measure of software physical or functional size. In contrast, LOC, Function count, and Halstead's Program Length are recognized size metrics.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 250, 422; companion slides 5, 98.

Q6.11. Which one is not a measure of software science theory?

(a) Vocabulary · (b) Volume · (c) Level · (d) Logic

(d) Logic

Halstead's Software Science suite includes Vocabulary (η), Length (N), Volume (V), Potential Volume (V^*), Program Level (L), Difficulty (D), Effort (E), Time (T), and Language Level (λ). There is no measure named "Logic" in Halstead's theory.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 255–262; companion slides 14–23, 98.

Q6.12. A human mind is capable of making how many number of elementary mental discriminations per second (i.e., Stroud number)?

(a) 5 to 20 · (b) 20 to 40 · (c) 1 to 10 · (d) 40 to 80

(a) 5 to 20

Halstead adopted psychologist John M. Stroud's perceptual research, which established that the human mind is capable of making between 5 and 20 elementary mental discriminations per second. In formulating practical Software Science equations, Halstead adopted the reference value $S = 18$ discriminations per second to convert mental effort E into estimated programming time in seconds:

$$T = \frac{E}{S} = \frac{E}{18} \text{ seconds}$$

Option (a) directly captures the empirical Stroud frequency range.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 260; companion slides 20, 98.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- ثابت ستروود (Stroud number) يحدد أن العقل البشري ينجز بين 5 و20 تمييزاً عقلياً في الثانية الواحدة. واعتمد هالستيد القيمة $S = 18$ في معادلات تحويل الجهد إلى زمن بالسكوند ($T = E/18$).

Q6.13. Minimal implementation of any algorithm was given the following name by Halstead:

(a) Volume · (b) Potential volume · (c) Effective volume · (d) None of the above

(b) Potential volume

Potential Volume V^* is the volume of the most compact, idealized version of the algorithm, typically conceived as a single call to a pre-existing function where only the unique input and output parameters are supplied. It is expressed as:

$$V^* = (2 + \eta_2^*) \log_2 (2 + \eta_2^*)$$

Where η_2^* represents the number of conceptually essential input/output operands.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 257; companion slides 15, 99.

Q6.14. Program volume of a software product is:

(a) $V = N \log_2 \eta$ · (b) $V = (N/2) \log_2 \eta$ · (c) $V = 2N \log_2 \eta$ · (d) $V = N \log_2 \eta + 1$

(a) $V = N \log_2 \eta$

Program volume is computed as total program length N multiplied by the binary logarithm of the vocabulary η:

$$V = N \log_2 \eta = (N_1 + N_2) \log_2 (\eta_1 + \eta_2)$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 257; companion slides 14, 99.

Q6.15. Which one is the international standard for size measure?

(a) LOC · (b) Function count · (c) Program length · (d) None of the above

No unique key can be verified from the supplied course item. The stem says "international standard for size measure," while the options are LOC, Function count, Program length, and None of the above. The source does not define what "international standard" means here or provide an answer key. Choice (b) may be the intended classroom answer if "Function count" is being used loosely for function-oriented sizing, but that interpretation is not established by the item itself.

Source item: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slide 99; scan p. 305. No publisher key located.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- السؤال وخياراته لا تكفي لإثبات مفتاح وحيد؛ احتمال (b) تفسير للنية فقط، وليس مفتاحاً مؤكداً.

Q6.16. Which one is not an object oriented metric?

(a) RFC · (b) CBO · (c) DAC · (d) OBC

(d) OBC

Response For a Class (RFC) and Coupling Between Objects (CBO) belong to the seminal Chidamber & Kemerer (CK) suite, and Data Access Component (DAC) belongs to the Li & Henry object-oriented metrics suite.

"OBC" is not a recognized metric in software engineering literature.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–293; companion slides 80–84, 99.

Q6.17. Which metric also consider indirect connected methods?

(a) TCC · (b) LCC · (c) Both of the above · (d) None of the above

(b) LCC

Bieman & Kang (1995) defined two cohesion metrics based on attribute sharing among methods:

- **Tight Class Cohesion (TCC): Measures the relative number of directly connected method pairs** (two methods are directly connected if they access at least one common instance variable).
- **Loose Class Cohesion (LCC): Measures the relative number of directly or indirectly connected method pairs** (two methods are indirectly connected if they are connected through a sequence of intermediate method accesses).

Therefore, $LCC \geq TCC$ always holds.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 292–293; companion slides 85, 100.

Q6.18. Depth of inheritance tree (DIT) can be measured by:

(a) Number of ancestors classes · (b) Number of successor classes · (c) Number of failure classes · (d) Number of root classes

(a) Number of ancestors classes

In companion slide 100 and textbook scan p. 305, Depth of Inheritance Tree (DIT) is defined as the number of ancestor classes (the maximum path length from the class node to the root class of the inheritance tree). Successor classes correspond to descendants/subclasses (measured by NOC - Number of Children).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 289; companion slides 81, 100.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- يقاس عمق شجرة الوراثة (DIT) بعدد الكلاسات السلفية (Number of ancestor classes) صعوداً إلى كلاس الجذر.
- في الوراثة المتعددة (Multiple Inheritance)، يؤخذ المسار الأطول إلى الجذر.

Q6.19. A dynamic page is:

(a) Where contents are not dependent on the actions of the user · (b) Where contents are dependent on the actions of the user · (c) Where contents cannot be displayed · (d) None of the above

> **(b) Where contents are dependent on the actions of the user**
>
> **Dynamic web pages are constructed on demand on the web server by executing server-side scripts and querying backend databases, where contents are dependent on the actions, requests, or session state of the user.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 91, 100.*

Q6.20. Which of the following is not a size metric?

(a) LOC · (b) FP · (c) Cyclomatic Complexity · (d) Program length

> **(c) Cyclomatic Complexity**
>
> **Cyclomatic Complexity is a control-flow structural complexity metric measuring linearly independent basis paths, not an attribute of software size.** In contrast, LOC, FP, and Halstead's Program Length are recognized physical or functional size metrics.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 5, 98, 100.*

Section 2: Syllabus Gap-Closing Synthesis MCQs (15 Items)

Q6.21. In measurement theory (Stevens' typology), which measurement scale possesses a non-arbitrary, true absolute zero point?

(a) Nominal scale · (b) Ordinal scale · (c) Interval scale · (d) Ratio scale

> **(d) Ratio scale**
>
> **A Ratio scale has ordered values, equal intervals, and a non-arbitrary zero. Ratios are meaningful on this scale; for example, 2,000 LOC is twice 1,000 LOC when both counts use the same rules.** An Interval scale has equal intervals but an arbitrary zero.
>
> *Source: Fenton, "Software Measurement: A Necessary Scientific Basis," pp. 200–201, https://doi.org/10.1109/32.268921; Aggarwal & Singh (3rd ed.), scan p. 252.*

Q6.22. Which of the following metrics is classified as a direct measure of an internal software attribute?

(a) System Reliability (Mean Time Between Failures) · (b) Developer Productivity (LOC per Person-Month) · (c) Source Code Length (Lines of Code) · (d) System Maintainability

(c) Source Code Length (Lines of Code)

Lines of Code (LOC) is a direct measure because it can be assessed strictly by examining the entity itself without calculating or observing other attributes. Furthermore, it measures an **internal attribute** because it depends solely on the source code artifact itself, independent of runtime behavior or the external operating environment. MTBF and Maintainability are external attributes; Productivity is an indirect process metric.

Source: Sommerville, I., "Software Engineering", 9th ed., printed pp. 668–671.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- المقياس المباشر (Direct Measure) يُقاس مباشرة من الكيان دون الحاجة لمعادلة تشتق قيمته من متغيرات أخرى.
- السمة الداخلية (Internal Attribute) تخص الكيان بمفرده (مثل حجم الكود LOC أو تعقيده)، بينما السمة الخارجية (External Attribute) تظهر فقط أثناء التشغيل والاستخدام (مثل الاعتمادية وسهولة الصيانة).

Q6.23. In the Goal–Question–Metric (GQM) paradigm, what is the correct hierarchical order from top to bottom?

(a) Conceptual Level → Operational Level → Quantitative Level · (b) Operational Level → Quantitative Level → Conceptual Level · (c) Quantitative Level → Conceptual Level → Operational Level · (d) Metric → Goal → Question

(a) Conceptual Level → Operational Level → Quantitative Level

The Goal–Question–Metric (GQM) framework is structured across three formal hierarchical levels:

1. **Conceptual Level (Goal): A high-level business or engineering objective is established for an object with respect to various quality models from a specific viewpoint.**
2. **Operational Level (Question): A set of specific questions is formulated to characterize the assessment or achievement of each goal.**
3. **Quantitative Level (Metric): A set of objective or subjective metrics is associated with each question to answer it quantitatively.**

Source: Leach, R., printed pp. 39–40.

Q6.24. According to Sommerville, what is the fundamental difference between predictor metrics and control metrics?

(a) Predictor metrics measure hardware; control metrics measure software · (b) Predictor metrics forecast future attributes; control metrics monitor active process progress · (c) Predictor metrics are subjective; control metrics are automated · (d) Predictor metrics apply only to Agile; control metrics apply to Waterfall

(b) Predictor metrics forecast future attributes; control metrics monitor active process progress

Software metrics fulfill two distinct management functions:

- **Predictor metrics: Measure current internal attributes to forecast future internal or external attributes of software** (e.g., using cyclomatic complexity and size to predict future defect density or maintenance cost).
- **Control metrics: Provide real-time visibility into an active process to allow managers to control development progress against plans** (e.g., tracking average days to resolve defect reports or schedule burn-down rate).

Source: Sommerville, I., 9th ed., printed pp. 668–670.

Q6.25. Which of the following is considered a primary technical criticism of Lines of Code (LOC) as a productivity metric?

(a) It cannot be counted automatically by tools · (b) It penalizes expressive, high-level languages and rewards verbose code · (c) It is not accepted by the IEEE · (d) It cannot be measured in procedural languages

(b) It penalizes expressive, high-level languages and rewards verbose code

Because LOC measures physical syntax rather than functional utility, developers writing in concise, high-level languages (e.g., Python, SQL) produce far fewer lines of code to achieve the exact same functionality as a verbose implementation in C or Assembly. Calculating productivity as LOC/PM creates a false paradox where verbose, bloated implementations appear to have higher productivity than clean, expressive solutions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–142; Pressman, 5th ed., printed pp. 92–95.

Q6.26. In Albrecht's Function Point Analysis, which of the following is an external functional component?

(a) Internal Logical Files (ILF) · (b) External Inquiries (EQ) · (c) General System Characteristics (GSC) · (d) Value Adjustment Factor (VAF)

(b) External Inquiries (EQ)

Albrecht's Function Point model identifies five primary information domain components:

1. **External Inputs (EI):** Elementary processes that process data entering from outside.
2. **External Outputs (EO):** Elementary processes that send derived data outside.
3. **External Inquiries (EQ):** Elementary input-output combinations that retrieve data immediately without updating files.
4. **Internal Logical Files (ILF):** User-identifiable groups of logically related data maintained entirely inside the application boundary.
5. **External Interface Files (EIF):** Files maintained by other systems referenced by the application.

EQ is an external transaction component. ILF is internal; GSC and VAF are adjustment factors, not information domain components.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 142–146; companion slides Ch.4.

Q6.27. What is the standard range of the Value Adjustment Factor (VAF) in Albrecht's Function Point calculation?

(a) 0.00 to 1.00 · (b) 0.65 to 1.35 · (c) 0.80 to 1.20 · (d) 1.00 to 2.00

(b) 0.65 to 1.35

The Value Adjustment Factor (VAF) is derived from 14 General System Characteristics (F_1 to F_{14}), each rated from 0 (no influence) to 5 (strong influence):

$$VAF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$$

- **Minimum possible VAF:** When all $F_i = 0$, $VAF = 0.65 + 0.01(0) = 0.65$ ($\pm 35\%$ adjustment downwards).
- **Maximum possible VAF:** When all $F_i = 5$, $\sum F_i = 70$, $VAF = 0.65 + 0.01(70) = 1.35$ ($\pm 35\%$ adjustment upwards).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 145; Ch.4 companion slides.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- يتراوح معامل تعديل القيمة (VAF) دوماً بين 0.65 و1.35، وهو ما يعطي تعديلاً أقصاه $\pm 35\%$ على النقاط الوظيفية غير المعدلة (UFP).

Q6.28. In Object Points sizing (as used in COCOMO II Early Prototyping), how are software objects identified?

(a) Number of C++ class definitions · (b) Number of screens, reports, and 3GL components · (c) Number of database tables and foreign keys · (d) Number of use cases and actors

(b) Number of screens, reports, and 3GL components

Object Points (also known as Application Points) is a high-level functional size metric computed during early prototyping before detailed code specifications exist. System size is quantified by counting screens, reports, and third-generation language (3GL) modules, classifying each as simple, medium, or difficult.

Source: Mall, R., "Fundamentals of Software Engineering", 4th ed., printed pp. 165–166.

Q6.29. In Halstead's Software Science, what is the mathematical relationship between Program Difficulty D and Program Level L?

(a) $D = L$ · (b) $D = 1/L$ · (c) $D = L^2$ · (d) $D = \log_2 L$

(b) $D = 1/L$

Program difficulty D is defined as the reciprocal of the program level L:

$$D = \frac{1}{L} = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$$

As the program level decreases (indicating greater implementation verbosity or complexity), the difficulty of comprehending or writing the program increases proportionally.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 259; companion slides 18–19.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- صعوبة البرنامج (D) هي المعكوس الدقيق لمستوى البرنامج (L): $D = 1/L$. صيغة حسابها العملية هي: $(\eta_1/2) \times (N_2/\eta_2)$.

Q6.30. In Henry & Kafura's Information Flow metric (as documented in Pressman 5e §19.4.2), what is the formula for the complexity of module i?

(a) $HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]$ · (b) $HKM = \text{length}(i) \times [fan_in(i) \times fan_out(i)]^2$ · (c) $HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$ · (d) $HKM = \frac{fan_in(i) + fan_out(i)}{\text{length}(i)}$

(c) $HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$

As verified in Pressman 5th ed. §19.4.2 (printed p. 524 / PDF p. 552), the Henry and Kafura Metric (HKM) computes module structural complexity as:

$$HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$$

Where $\text{length}(i)$ is the module size in lines of code, $fan_in(i)$ includes control connections and data structures retrieved, and $fan_out(i)$ includes modules called and data structures updated. The squared term represents the non-linear potential communication paths.

Source: Pressman, R., "Software Engineering: A Practitioner's Approach", 5th ed., printed p. 524.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- معادلة هنري وكافورا المعتمدة في مرجع Pressman §19.4.2 هي: الحجم مضروب في مربع مجموع (fan-in + fan-out). التربيع يمثل النمو غير الخطي لقنوات الاتصال المحتملة.

Q6.31. For a strongly connected control flow graph with E edges, N nodes, and P connected components, McCabe's Cyclomatic Complexity $V(G)$ is:

(a) $V(G) = E - N + P$ · (b) $V(G) = E - N + 2P$ · (c) $V(G) = N - E + 2P$ · (d) $V(G) = E + N - P$

(b) $V(G) = E - N + 2P$

Thomas McCabe defined cyclomatic complexity using graph theory for a control flow graph G as:

$$V(G) = E - N + 2P$$

For a single single-entry, single-exit module where $P = 1$, the formula reduces to $V(G) = E - N + 2$. It can also be computed as the number of predicate (decision) nodes plus one ($P_{pred} + 1$), or the number of bounded enclosed regions plus one.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–423; companion slides Ch.8.

Q6.32. In the Chidamber & Kemerer metric suite, Lack of Cohesion in Methods (LCOM) quantifies:

(a) The percentage of lines that contain comments · (b) The number of method pairs that do not share instance variables minus the number that do · (c) The depth of inheritance · (d) The number of parameters per method

(b) The number of method pairs that do not share instance variables minus the number that do

LCOM measures the dissimilarity of methods in a class based on shared attribute access. Let P be the number of method pairs that share no instance variables, and Q be the number of method pairs that share at least one instance variable. Then:

$$LCOM = \begin{cases} P - Q & \text{if } P > Q \\ 0 & \text{otherwise} \end{cases}$$

A high LCOM indicates low cohesion, suggesting that the class performs disparate tasks and should be refactored into smaller subclasses.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 292–293; companion slides 84–85.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- مقياس LCOM يقيس ضعف أو غياب التماسك، فكلما كبرت قيمته دل ذلك على سوء التصميم وتشتت الكلاس، وإذا كان عدد الأزواج المشتركة أكبر من غير المشتركة يُسند الصفر ($LCOM = 0$).

Q6.33. Which expression gives the unadjusted Use Case Point total using the course actor and use-case weights?

(a) $UUCP = UAW + UUCW$ · (b) $UUCP = UAW \times UUCW$ · (c) $UUCP = TCF + ECF$ · (d) $UUCP = LOC/PM$

(a) $UUCP = UAW + UUCW$

The unadjusted total is the sum of the weighted actor count (UAW) and weighted use-case count ($UUCW$). The inspected course material does not establish additional adjustment factors or an effort conversion.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 295–296; companion slides 89–90.

Q6.34. Defect Removal Efficiency (DRE) is mathematically defined as:

(a) $DRE = E/(E + D)$ · (b) $DRE = D/(E + D)$ · (c) $DRE = (E + D)/E$ · (d) $DRE = E \times D$

(a) $DRE = E/(E + D)$

Defect Removal Efficiency (DRE) measures the quality filtering ability of software review and testing activities. It is defined as:

$$DRE = \frac{E}{E + D}$$

- E**: Number of defects discovered during the software development phase (before release).**
- D**: Number of defects discovered by customers or downstream testing phases (after release).**

A DRE approaching 1.0 (or 100%) indicates that virtually all defects were caught and eradicated internally prior to delivery.

Source: Pressman, R., 5th ed., printed p. 98 / PDF p. 126; Aggarwal & Singh, companion slide 10.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- كفاءة إزالة العيوب (DRE) هي نسبة الأخطاء المكتشفة داخلياً قبل التسليم (E) إلى إجمالي الأخطاء ($E + D$). كلما اقتربت من 1.0 كانت اختبارات ومراجعات الفريق مثالية.

Q6.35. Which organizational pitfall describes developers deliberately changing their work habits to inflate a tracked metric at the expense of true software quality?

(a) Brooks' Law · (b) Measurement Dysfunction / Gaming · (c) Pareto Principle · (d) Software Entropy

(b) Measurement Dysfunction / Gaming

Metric gaming is a possible form of measurement dysfunction: when a narrow measure becomes an incentive target, people may optimize the reported number instead of the intended outcome. For example, a LOC bonus could encourage unnecessary code, while a closed-ticket target could encourage prioritizing easy tickets. These are possible scenario risks, not guaranteed effects or claims attributed to a named law.

Source: Aggarwal & Singh, companion slides 7–8, for implementation problems; scenario examples are authored.

Verify distinguishing criteria between Halstead, McCabe, Information Flow, and Object-Oriented metric families.